

无线电干扰源快速自动定位与清除：几何保证搜索与强化学习调度

摘要

本文针对 2026 年高教社杯全国大学生数学建模竞赛 B 题，建立了一套以可证明发现覆盖、集合定位和有限步骤清除为硬保证，以强化学习调度为可选上层策略的机器狗自动定位清除方案。方案将模型分成两层：保证层负责覆盖证书、定位可行域、清除证书和光学保底；决策层在有限合法动作中选择下一步，不影响不漏源的完成保证。

针对问题 1，将 1 度示向误差写成三个半平面约束，利用 Sutherland-Hodgman 逐边裁剪求凸多边形定位区域，枚举顶点对求直径，并用 Welzl 算法求最小包围圆。数值验证表明：边长 40 米等边三角形直径 40 米，但最小包围圆半径约 23.09 米，因此直径不超过 40 米并不等于一次清除必成功。

针对问题 2，首次示向给出可行域 F ，第二点必须同时保证可接收和具有较好交会几何。本文给出保证第二点在最小接收半径 1000 米内的候选区域 Q_{vis} ，并证明 $q_{\pm} = s_1 + 750u \pm 300v$ 的统一最坏距离上界约 817.65 米，小于 1000 米；进一步采样最坏约 807.7 米。

针对问题 3，构造原点加半径 1200 米正六边形共 7 个测点 S_3 ，证明任意全向源到某个测点距离不超过约 968.90 米，必被发现。针对问题 4，构造原点、内环 12 点、外环 12 点共 25 个测点 S_{25} ，将区域剖分为 36 个三角形，最大边长 975.897 米小于最小接收半径 1000 米；任意源位于某三角形内，任意 180 度发射半平面至少包含一个锥顶点，因此任意位置、任意方向的定向源至少有一个测点可接收。 S_{25} 完整覆盖路线约 18173.85 米；原 31 点网格任意完整访问路线至少 28500 米，固定覆盖巡航路线至少缩短 36.2%。

单源清除采用双侧区间定位：第一次有效示向后，真实源满足 $0 \leq x \leq 1500$ 、 $|y| \leq x \cdot \tan(1.01^\circ)$ 。在区间中点两侧成对测量，两次 no_signal 可安全推出 x 小于中点。每轮区间至少折半，最多 12 次追加测向；末端用两个光学点覆盖剩余矩形，最坏距离约 17.856 米小于 20 米，因此最多 2 次末端光学尝试即可保证清除。原有 122 点光学扫描保留为最终保底。

在调度层，将任务建模为部分可观测半马尔可夫决策过程，采用 96 槽位宏动作和带动作屏蔽的 PPO。奖励直接对齐虚拟总时间 $r_t = -\Delta T_t / 100$ 。由于本地与官方实验均显示 PPO 在全部决策中未偏离 A_0 先验，本文将 PPO 定位为安全调度扩展，而将严格几何主动搜索 G25OR 作为最终主方案。G25OR 同时修复了首次示向三角形约 0.233 米的外包缺口，并使用最小包围圆作为清除证书。

官方模拟器实验方面，G25O、G25OR、G25O-R full 三个版本各完成问题 3、问题 4 各 100 局演练，全部局数分别为 100/100、100/100、100/100 清除。G25OR 问题 3、问题 4 平均定位清除时间分别为 328.81 秒/源和 556.83 秒/源，平均程序运行时间分别为 1.462 秒和 3.548 秒；相对更早的旧安全 PPO 分别降低约 23.3% 和 50.4%，相对传统方法有效演练组分别降低约 44.7% 和 49.9%。三版本之间两两差异均不显著 ($p > 0.17$)，说明性能主要来自几何保证层而非 PPO 网络。

关键词：交会定位；半平面求交；最小包围圆；25 点覆盖；双侧区间定位；动作屏蔽；PPO；光学保底

一、问题重述

本文研究 2026 年高教社杯全国大学生数学建模竞赛 B 题[1]。目标区域为半径 1800 米的圆形区域，原点为圆心，正东为 x 轴正向，正北为 y 轴正向。区域内存在 10 至 16 个干扰源，具体个数未知；每个源占用 1 至 20 中的一个频道，不同源频道互不相同。每个源有效接收半径在 1000 至 1500 米之间。全向源在 360 度范围内可接收；定向源只在某方向两侧各 90 度内发射。测向机示向度误差不超过 1 度，同一地点电磁环境固定，重复测量不能平均掉该误差。

机器狗从原点出发，初始频道为 1，移动速度 5 米/秒。一次 /measure 的虚拟耗时为移动距离/5、频道切换（每次 1 秒）和 5 秒检测时间；一次 /clear 的耗时为移动距离/5 加 3 秒（未发现）或 5 秒（成功清除）。/clear 不改变测向机当前频道。任务要求在确保全部源被清除的前提下，使总虚拟时间尽可能短，并关心清除率和平均定位清除时间。

题目要求给出：问题 1 的定位区域直径算法；问题 2 的第二检测点选择策略；问题 3 全向多源搜索定位清除策略；问题 4 混合定向源策略；并通过官方模拟器演练和正式测试检验。

1.3 总体技术路线

本题的困难不在于机器人运动学，而在于源数量、位置、频道、半径、类型和定向方向全部未知。若把完成率完全交给强化学习奖励，容易出现漏源；若只用固定巡航，则移动时间过长。因此本文采用三层结构：

保证层：S3/S25 保证覆盖、频道状态账本、完成证书、122 点光学保底。该层保证只要按流程执行，有限步骤内必不漏源。

几何层：示向误差半平面交、最小包围圆清除证书、双侧区间定位、25 点方向覆盖。该层决定需要走哪些点、何时可以清除。

决策层：A0 确定性策略、带动作屏蔽 PPO、安全门。该层只影响效率，不影响正确性。

总体技术路线如图 1 所示。该框架把正确性保证与效率优化分离：先由几何层给出不漏源、可清除的硬条件，再由决策层在合法动作集合内压缩移动和测量开销。

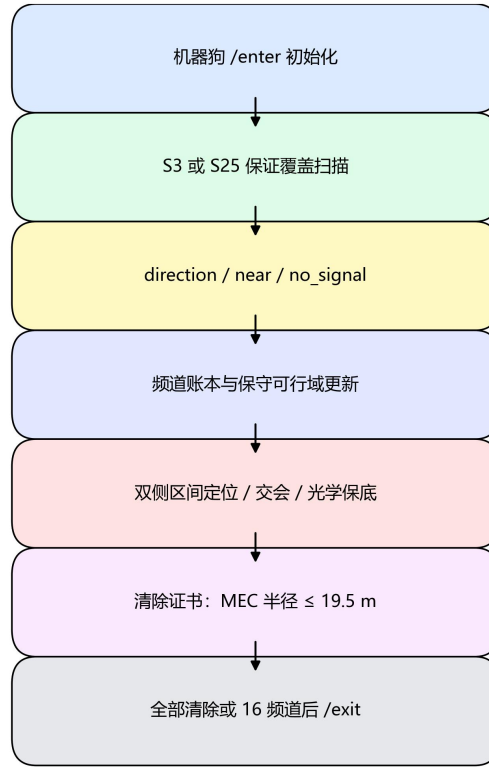


图 1 总体技术路线框架

虚拟总时间统一表示为

$$T = L/5 + 5N_{\text{meas}} + N_{\text{sw}} + 3N_{\text{fail}} + 5N_{\text{succ}}, \quad (1)$$

其中 L 为总移动距离， N_{meas} 、 N_{sw} 、 N_{fail} 、 N_{succ} 分别为测量、切频、失败清除和成功清除次数。后文所有表格和优化目标均使用该时间口径。各问题候选算法与选择理由集中见第十一节。

二、问题分析

2.1 整体任务与数学类型

题目要求同时完成四项任务：给出定位区域直径算法；给出第二测点选择策略；设计全向多源搜索定位清除方案；设计混合定向源的搜索定位清除方案。从数学上看，问题 1 是带硬角度约束的凸集几何计算，问题 2 是保证第二测点可接收前提下的效用优化，问题 3、4 则是未知集合覆盖、单源集合定位与在线部分可观测决策的耦合问题。四个任务共享同一套测量—定位—清除数据流，前序问题的输出会直接成为后续问题的输入，因此需要统一建模而不是四个孤立模块的简单拼接。

2.2 问题之间的数据流与逻辑联系

问题 1 提供的半平面交、直径和最小包围圆算子，是问题 2 评估候选测点、问题 3、4 构造清除证书的几何基础。问题 2 在首次示向后选择第二测点，一方面保证该点仍处于源的有效接收范围内，另一方面为后续双侧区间定位提供具有足够交会角的观测。问题 3 建立 S3 发现覆盖、频道账本和完成证书；问题 4 在其上增加方向性约束，用 S25 方向覆盖保证任意位置、任意 180 度发射方向的定向源至少被一个覆盖点接收。单源清除阶段再次复用问题 1 的可行域算子：前两次有效示向给出可行域，双侧区间测量逐步折半，最小包围圆证书判断一次清除是否能够覆盖整个可行域。由此形成覆盖发现、频道筛选、示向定位、清除验证和完成证书的在线闭环。

2.3 附件与数据特点

本题附件给出的是模拟器通信接口、计时规则和物理边界，不提供历史观测数据，因此不存在传统意义上的缺失值和异常值清洗。数据特点体现在四个方面：一是源数量、位置、频道、半径和定向方向均隐藏，只能在在线交互中逐步揭示；二是角度误差为题目给定的硬界，而非已知概率分布；三是测量结果包括 `direction`、`near` 和 `no_signal` 三类，其中 `no_signal` 只说明当前位置不可接收，不能直接推出源不存在；四是官方模拟器案例随机生成且不可重放，本地实验则可通过固定种子实现严格配对。因此，预处理重点不是数据平滑，而是把题目硬界转化为保守可行域：角度误差取 1.01 度，接收半径取下界 1000 米，清除半径取 20 米，并用 1.01 度的正切构造外切安全区域。

2.4 求解框架与检验思路

总体技术路线如图 1 所示。正确性由三层保证：S3 与 S25 保证发现覆盖，频道账本保证不遗漏，最小包围圆与双侧定位保证清除。效率由 A0、动作屏蔽 PPO 和安全门组成的调度层优化，但调度层只改变动作顺序，不改变有限步骤完成路径。模型检验采用解析证明、数值采样、本地严格配对、官方演练、非参数检验与成本敏感性分析相结合：解析证明给出最坏界，20 万级采样检验边界与漏检，相同种子与误差场的本地配对控制混杂因素，官方 100 局分布比较验证工程稳定性，Mann-Whitney 检验避免把非配对差值写成确定增益。正式测试尚未执行，因此第 10.7 节的记录表暂不填写，待组委会给出正式案例后按冻结参数运行。

三、模型假设与符号说明

3.1 模型假设

所有检测点、源和机器狗均在二维平面内，忽略高度差。

/measure 返回 direction 时，真实源位于以该示向度为中心、半角 1 度的前向楔形内；返回 near 时源在检测点 5 米内；返回 no_signal 时只表示当前位置不可接收，不直接等价于源不存在。

/clear 成功仅取决于清除点与对应频道源的距离是否不超过 20 米，与定向源发射方向无关。

同一地点、同一频道的示向误差固定；实现中取保守误差界 $\epsilon=1.01$ 度，包含题目 1 度误差和两位小数舍入余量。

目标圆域、最大接收半径 1500 米和最小接收半径 1000 米为硬约束。

本地实验分布是设计者设定，不是官方生成分布；官方正式测试尚未执行。

3.2 主要符号

本文反复使用的主要符号及其含义、单位见表 1。

表 1 主要符号表

符号	含义	单位
s_i	第 i 个检测点	m
θ_i	第 i 次示向度	度
ϵ	示向误差上界	度
W_i	由第 i 次测量得到的误差楔形	-
P, P_i	定位区域或频道 i 的保守可行域	-
$D(P)$	定位区域直径	m
r^*	最小包围圆半径	m
$q_{\pm}$	第二测点可靠候选	m
S_3	问题 3 的 7 个覆盖点	-
S_{25}	问题 4 的 25 个覆盖点	-
$N_{\text{meas}}, N_{\text{sw}}, N_{\text{fail}}, N_{\text{succ}}$	测量、切频、失败清除、成功清除次数	次
T	虚拟总时间	s
b_t, a_t, r_t	决策时刻的账本状态、宏动作、奖励	-

四、问题 1：几何定位模型

4.1 示向误差的半平面表示

设检测点 $s_i=(x_i, y_i)$ ，示向度 θ_i ，误差界 $\varepsilon=1$ 度。定义沿示向方向单位向量 $u_i=(\cos\theta_i, \sin\theta_i)$ ，垂直向量 $v_i=(-\sin\theta_i, \cos\theta_i)$ 。真实位置 z 必须满足前向条件

$$u_i^T(z-s_i) \geq 0 \quad (2)$$

以及角度偏差条件

$$|v_i^T(z-s_i)| \leq \tan\varepsilon \cdot u_i^T(z-s_i)。$$

将绝对值展开，可得三个半平面：

$$-u_i^T z \leq -u_i^T s_i, \quad (3)$$

$$(v_i \tan\varepsilon u_i)^T z \leq (v_i \tan\varepsilon u_i)^T s_i, \quad (4)$$

$$(-v_i \tan\varepsilon u_i)^T z \leq (-v_i \tan\varepsilon u_i)^T s_i。 \quad (5)$$

记三个半平面的交集为 W_i 。多次测量的定位区域为

$$P = \cap_i W_i。 \quad (6)$$

P 为凸多边形或无界凸集。实际计算中加入目标圆域 $B(0, 1800)$ 和单次接收距离上界 $B(s_i, 1500)$ 的保守外切多边形近似，保证近似集合包含真实集合。

4.2 定位区域直径算法

用 Sutherland-Hodgman 算法逐边裁剪凸多边形：初始多边形取目标圆域外切正 128 边形；对每个半平面 $a x + b y \leq c$ ，保留满足约束的顶点并求边界交点。得到 P 的顶点集合 $V(P)$ 后，区域直径为

$$D(P) = \max_{a, b \in V(P)} \|a - b\|。 \quad (7)$$

第一版枚举顶点对，复杂度 $O(n^2)$ 。本文同时实现旋转卡壳算法作为交叉验证，随机凸多边形测试中两者最大差 0.0 米。

4.3 直径、覆盖圆与最小包围圆

设 $r^* = \min_q \max_{z \in P} \|q - z\|$ 为定位区域最小包围圆半径。一次 /clear 能保证成功的条件是 $r^* \leq 20$ 米；工程实现取 $r^* \leq 19.5$ 米，留 0.5 米数值余量。

需要注意：直径 $D(P) \leq 40$ 米并不等价于一次清除必成功。定义以定位区域直径作直径的圆可能无法覆盖整个区域。例如边长 40 米的等边三角形， $D=40$ 米，但 $r^* = 40/\sqrt{3} \approx 23.09$ 米，大于 20 米，不能保证一次清除。

本文用 Welzl 随机增量算法[7]求最小包围圆，复杂度期望 $O(n)$ 。若圆半径满足阈值，则圆心即为可靠清除点。由上述例子可知，以定位区域直径作直径的圆不一定覆盖定位区域；只有最小包围圆半径不超过 20 米时，才构成一次清除证书。特殊情形包括空集、单点、线段、近似共线、重复顶点和无界交集，均单独处理。

4.4 算法步骤

算法 1：定位区域直径与清除证书。

输入：检测点 s_i 、示向度 θ_i 、误差界 ε 。

步骤 1：由每个观测构造三个半平面。

步骤 2：初始化目标圆域外切正多边形。

步骤 3：逐半平面执行 Sutherland-Hodgman 裁剪；若中间为空终止。

步骤 4：枚举顶点对求直径；同时用旋转卡壳交叉验证。

步骤 5：求最小包围圆；若 $r^* \leq 19.5$ ，则输出清除证书，否则继续测量。

五、问题 2：第二测点选择模型

5.1 首次示向可行域

设首次检测点为 s_1 ，示向度为 θ_1 。全向源在距离不超过 1500 米时可能被接收，因此首次示向给出的保守可行域为

$$F = D \cap W_1 \cap B(s_1, 1500), \quad (8)$$

其中 $D=B(0,1800)$ 为目标圆域。为避免把 near 当作方向数据，可从 F 中排除距 s_1 不超过 5 米的区域。

5.2 保证第二测点可接收的候选区域

题目规定每个源有效接收半径至少 1000 米。对全向源，要保证第二测点 q 仍能收到信号，需要对 F 中最不利真实位置也保持距离不超过 1000 米，因此定义

$$Q_{\text{vis}} = \{ q : \max_{z \in F} \|q - z\| \leq 1000 \}. \quad (9)$$

只要 $q \in Q_{\text{vis}}$ ，即使实际接收半径取最小值 1000 米，第二点也不会因距离过远失联。

5.3 两个可直接使用的候选点

令 u 、 v 分别为首次示向方向和其垂直方向，取

$$q_{\pm} = s_1 + 750u \pm 300v. \quad (10)$$

对首次测量允许的整个扇区，可以证明 $\min(\|z - q_+\|, \|z - q_-\|)$ 不超过 q_+ 的最坏距离 817.65 米，因此一定小于 1000 米。进一步对扇区做密集网格和随机采样，最小距离最坏值约 807.7 米。 $q_{\pm}$ 是不需要知道真实距离的可证明接收候选，但不是全局最优点。

5.4 候选点评分

在 Q_{vis} 内可以生成更多候选点，并按单位时间定位不确定性下降排序。设当前可行域 F 、候选测点 q 、移动到 q 并测量一次的耗时

$$\tau(q) = \|q - p_d\| / 5 + 5 + 1\{c \neq c_d\}, \quad (11)$$

用设计者设定的场景样本 ξ_k 估计测量后可行域半径，定义

$$G(q) = [r^*(F) - E(r^*(F \cap W(q, \theta)))] / \tau(q), \quad (12)$$

选择 $G(q)$ 最大的候选点。样本只用于选点，清除保证仍由完整保守可行域给出。

六、问题 3、4：发现覆盖与单源清除保证

6.1 问题 3 的 7 点发现覆盖

取

$$S_3 = \{(0,0)\} \cup \{1200(\cos(k\pi/3), \sin(k\pi/3)) : k=0, \dots, 5\}, \quad (13)$$

即原点加半径 1200 米正六边形六个顶点。若源距原点不超过 1000 米，则原点可检测；否则源距原点 $r \in [1000, 1800]$ ，其极角与某六边形顶点方向夹角不超过 30 度，该顶点到源距离满足

$$d \leq r + 1200 - 2 \cdot 1200 \cdot r \cos 30^\circ. \quad (14)$$

右端为凸二次函数，在区间端点取最大，最大距离约 968.90 米小于 1000 米。因此任意全向源在 S_3 中至少一个测点可接收。数值采样 20 万点得到最坏 968.89 米，与解析值一致。

6.2 问题 4 的 25 点方向覆盖

问题 4 存在任意方向 180 度定向源。构造

$$S_{25} = \{O\} \cup \{I_k : k=0, \dots, 11\} \cup \{E_k : k=0, \dots, 11\}, \quad (15)$$

其中

$$I_k = 970(\cos(15^\circ + 30^\circ k), \sin(15^\circ + 30^\circ k)), \quad (16)$$

$$E_k = 1880(\cos(30^\circ k), \sin(30^\circ k)). \quad (17)$$

外环正十二边形内切圆半径为 $1880 \cos 15^\circ \approx 1815.94$ 米，大于 1800 米，覆盖整个目标圆。原点、内环和外环把覆盖区域剖分为 36 个三角形，最大边长 975.897 米小于最小接收半径 1000 米。

任意源 g 位于某个三角形内，到该三角形三个顶点距离都不超过 975.897 米。设定向源发射方向单位向量为 n ，则可见半平面为 $n^T(s-g) \geq 0$ 。因为 g 是三个顶点的凸组合，三个顶点不可能全部位于该半平面的严格背面，所以至少一个顶点既位

于有效发射侧，又在 1000 米以内。因此 S25 对任意位置、任意 180 度方向都能发现源。

完整的固定覆盖开路路线可取为 $O \rightarrow I_0 \rightarrow I_1 \rightarrow \dots \rightarrow I_{11} \rightarrow E_0 \rightarrow \dots \rightarrow E_{11}$ ，长度约 18173.85 米。原 31 点三角网格边长 950 米，任意完整访问 31 点的路线至少 $30 \times 950 = 28500$ 米。因此 S25 固定覆盖巡航路线至少缩短 36.2%。本文保留 S31 作为旧方案 and 对照。

S25 覆盖点与开路路线如图 2 所示。内环 12 点与外环 12 点错开半个扇区，使 36 个三角形的最大边长小于 1000 米；原点用于覆盖圆心附近并承担初始测量。

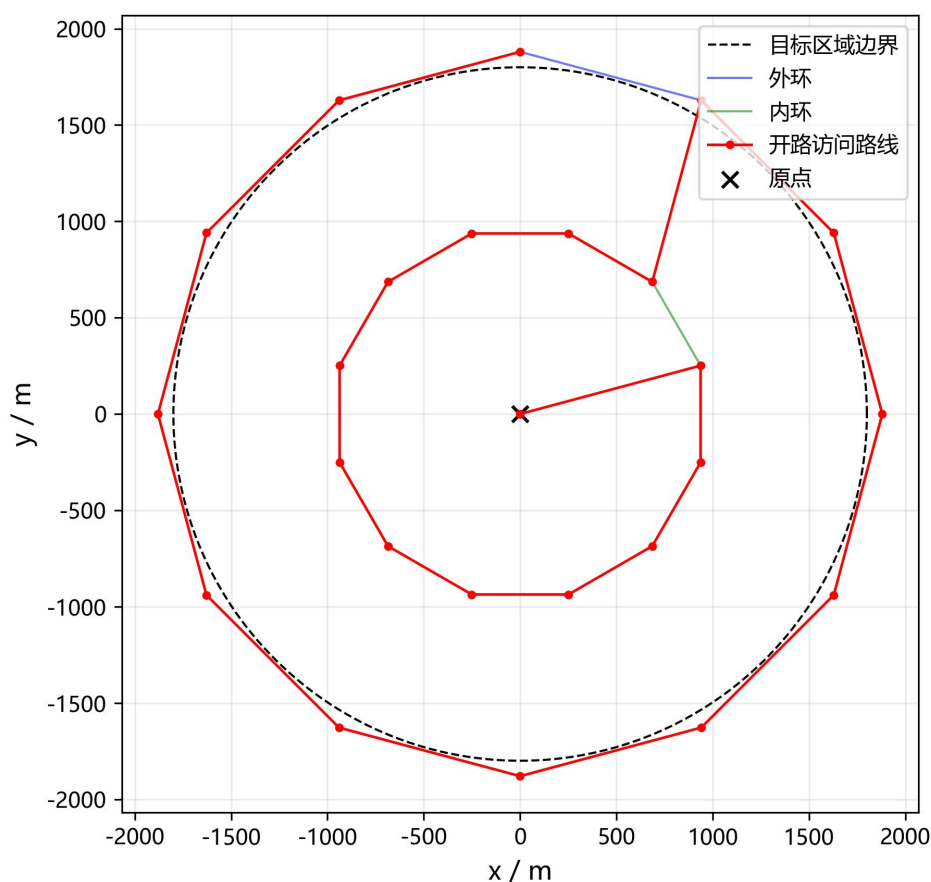


图 2 S25 覆盖点与开路路线

6.3 频道账本与完成证书

为 20 个频道分别维护状态：未发现、已发现未清除、已清除、已证实不存在。扫描某点时，检测所有仍有扫描义务的未发现频道；已发现频道转入局部定位；已清除频道跳过。若未发现频道在全部保证覆盖点上均无信号，由 S3 或 S25 覆盖证明可判定不存在。

程序只有在以下证书成立时才允许退出：已经成功清除至少 16 个不同频道；或 20 个频道均为已清除或已证实不存在。清除 10 个、连续无信号、粒子耗尽均不是合法停止条件。

6.4 双侧区间定位

首次有效示向后，以检测点为原点，示向方向为局部 x 轴。由误差界和最大接收半径，真实源位于

$$0 \leq x \leq 1500, |y| \leq x \cdot \tan(1.01^\circ). \quad (18)$$

维护 x 区间 $[l, u]$ 。当宽度大于 24 米时，取中点 $a = (l+u)/2$ ，侧偏 $b = a \cdot \tan(1.01^\circ) + 5$ ，依次测量 (a, b) 与 $(a, -b)$ 。

关键命题：两个探测点都返回 `no_signal` 时，必有 $x < a$ 。

证明思路：若 $x \geq a$ ，首次测点与源连线与直线 $x=a$ 的交点位于两条侧向探测点之间，该交点处于定向源可见半平面内；同时两个侧向点都比首次测点更接近源，不可能因距离过远而同时失联；若两点同时不可见，则它们之间的交点也不可见，与首次可见矛盾。

任一侧有 `direction` 时，结合误差界更新区间；收到 `near` 时直接清除。每轮区间至少折半，6 轮后宽度不超过 $1500/64 = 23.4375$ 米。末端用两个清除点

$$c_{\pm} = ((l+u)/2, \pm u \cdot \tan(1.01^\circ)/2) \quad (19)$$

覆盖剩余矩形，任意可能位置到最近点距离不超过 $\sqrt{12^2 + (1500 \tan 1.01^\circ / 2)^2} \approx 17.856$ 米小于 20 米。因此单源最多 12 次追加测向加 2 次末端光学尝试即可保证清除。

原有 122 点光学扫描仍保留为最终保底，当交会几何退化、接口异常或数值问题时使用。

七、方案 B：强化学习调度层

7.1 部分可观测半马尔可夫决策

真实源数量、频道、位置、半径、类型和方向均不可直接观测，机器人只能通过 `/measure` 和 `/clear` 返回逐步获取信息，因此任务为部分可观测决策问题。一次宏动作可能包含多次原子测量或多次清除，持续时间不同，因此用部分可观测半马尔可夫决策过程[5]描述，宏动作的实际虚拟耗时直接计入奖励。

7.2 状态空间

策略网络输入由确定性账本压缩得到，包括：机器人当前位置、当前测向机频道、累计虚拟时间、问题模式；20 个频道的状态独热、`direction/near` 标志、观测次

数、局部额外测量次数、是否已启动光学保底、保守可行域最小包围圆半径和中心、已扫描覆盖点数、失败清除次数、类型提示、发现后 no_signal 次数、最近结果编码；以及全局已清除、不存在、已发现和未解决频道数等。

状态中不输入真实源数量、真实位置、真实接收半径或真实定向方向。

7.3 动作空间与动作屏蔽

使用 96 个动作槽位：31 个 SCAN、16×3=48 个 REFINE、16 个 RESOLVE、1 个 EXIT。动作屏蔽禁止操作已清除频道、无首测依据的局部测量、无扫描义务的覆盖点，以及不满足完成证书的退出。动作屏蔽的实现要点与无效动作掩码 PPO 一致[6]；每个覆盖点的发现任务只执行一次；每个已发现源最多 6 次额外局部测量；额度耗尽后只允许可靠清除或光学保底。A0 启发式动作始终保留在合法动作集合中。

7.4 奖励函数

基础奖励直接对齐虚拟总时间：

$$r_t = -\Delta T_t / 100, \quad (20)$$

其中 ΔT_t 为宏动作实际消耗的虚拟时间。有限回合采用 $\gamma=1$ ，因此

$$\sum_t r_t = -T / 100. \quad (21)$$

为缓解稀疏奖励，可加入由可观测账本构造的势函数差，正常终止时势函数为 0，不改变总时间目标。

7.5 PPO 与安全门

采用带动作屏蔽的 actor-critic 网络，输出 96 维动作 logits 和状态价值。用 PPO[4]更新，初始学习率 3×10^{-4} ，截断参数 0.2，GAE 参数 0.95，小批量 256，每批 2048 个宏动作，更新 5 轮。

为在早期保持完成率，引入 A0 先验和行为克隆正则，并使用安全门：当 PPO 选择的动作不是 A0 先验且概率优势不足时，采用 A0 动作；异常或现实时间紧张时完全切换到 A0。因此该策略应称为带几何保证、光学保底和 A0 安全门的 PPO 混合调度，而不是把完成率全部归功于 PPO。

7.6 训练结果与定位

本地 100 个配对场景中，安全 PPO 与 A0 在问题 3 和问题 4 上平均虚拟时间完全相同；40 局官方演练中 PPO 偏离 A0 先验 0 次。这说明当前版本中 RL 网络没有独立产生速度收益，完成保证主要来自几何覆盖、清除证书和光学保底。因此本文

将 PPO 作为可选调度扩展和消融对象，最终主方案采用确定性几何主动搜索 G25OR。

八、G25OR 算法设计

8.1 严格外包首次示向扇形

首次收到 direction 后，以检测点 p 为顶点、示向角 θ 为中心构造保守三角形。若直接取边界射线长度 1500 米，远边弦在中央方向只覆盖到 $1500\cos1.01^\circ=1499.767$ 米，存在约 0.233 米的极薄缺口。G25OR 改为

$$L=1500/\cos1.01^\circ, \quad (22)$$

三角形顶点取

$$p, p+L\cdot u(\theta-1.01^\circ), p+L\cdot u(\theta+1.01^\circ)。 \quad (23)$$

该三角形严格包含半径 1500 米、半角 1.01 度的整个扇形。随机验证 167358 个目标圆内有效扇形点，0 个漏出保守三角形。

8.2 双侧区间定位与光学保底

对每个已发现源调用第 6.4 节的区间定位模块：最多 12 次追加测向和 2 次末端光学尝试。局部定位失败或失联时，回退到一次有效示向后的 122 点光学保底。

8.3 MEC 清除证书

清除证书使用最小包围圆，而不是包围盒中心与顶点最大距离。当保守可行域最小包围圆半径 ≤ 19.5 米时，认为该源可以一次可靠清除；清除点优先选在清除邻域内使路线绕行最小的位置。

8.4 滚动插入清除

扫描途中若某源已有清除证书，计算清除点在当前点 A 与下一覆盖点 B 之间的绕行增量

$$\Delta L=d(A,q)+d(q,B)-d(A,B), \quad (24)$$

若 $\Delta L \leq 300$ 米，则立即插入清除。该策略减少后续专程折返，同时不破坏任何完成保证。

8.5 机会式补测与动态调度

G25O/G25OR 在扫描点同时检查所有已发现源：若当前点到估计源的距离不超过约 1200 米，且当前观测与历史示向形成足够交会角，则顺手追加一次测量。完整滚动版 G25O-R 还实现了动态选择下一覆盖点、Q3 负观测圆盘无源证书和更激进的插入策略；但官方与本地实验均未显示显著收益，因此不作为正式方案。

九、算法流程与工程实现

9.1 主流程

算法 2：机器狗在线运行流程。在线运行流程如图 3 所示，其中安全门在策略输出越界或状态异常时回退到 A0 动作。

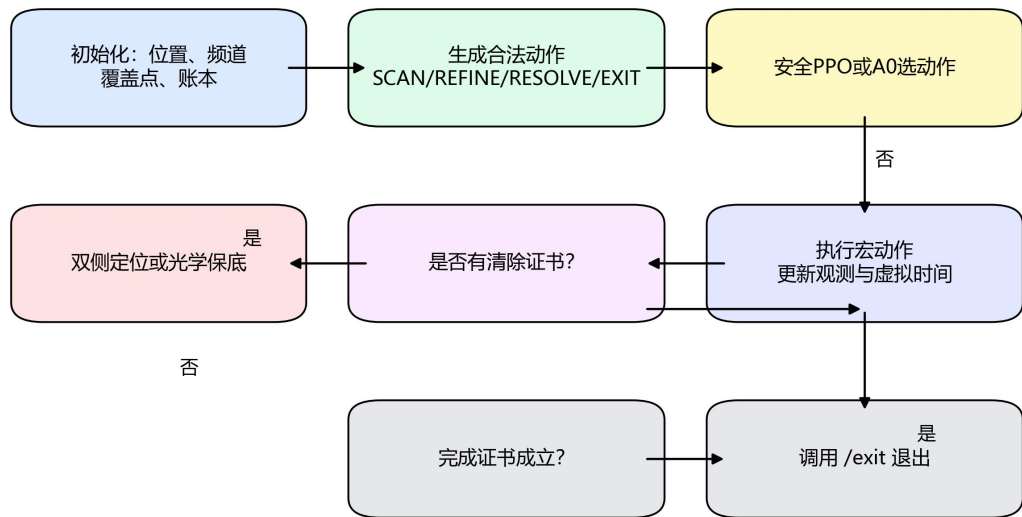


图 3 在线运行流程

步骤 1：调用 `/enter`，读取剩余现实时间，初始化频道账本、覆盖点集合和机器人状态。

步骤 2：若完成证书成立，调用 `/exit`。

步骤 3：根据全部历史反馈更新频道状态、保守可行域、扫描记录和辅助假设。

步骤 4：生成 96 槽位合法动作；A0 动作始终保留。

步骤 5：使用安全 PPO 或 A0 选择动作；异常时切换 A0。

步骤 6：串行执行动作：SCAN 在覆盖点检测所有需要发现的频道；REFINE 执行局部测量；RESOLVE 执行可靠清除或光学保底；EXIT 在证书成立后退出。

步骤 7：保存请求、响应、虚拟时间和统计信息，返回步骤 2。

9.2 官方接口适配

适配器严格按附件 1[2]和附件 2[3]实现 `/enter`、`/measure`、`/clear`、`/exit` 四个 POST 接口，使用 `arena_id="default"`、`robot_id` 和 `request_id`。每个新动作使用新 `request_id`；网络重试时复用原请求内容和原 `request_id`；逐次等待响应，不并发的

送不同动作；同时检查 HTTP 状态和 accepted 字段；使用 /enter 返回的 remaining_real_duration_s。

十、实验与结果分析

10.1 数学校验

数学校验结果见表 2。

表 2 数学校验结果

校验项	方法或样本	结果	判定
S3 覆盖最坏距离	解析	968.90 m	小于 1000 m，通过
S3 覆盖采样	20 万点	968.89 m，漏检 0	通过
S25 覆盖	36 个三角形	最大边 975.897 m	小于 1000 m，通过
S25 方向覆盖	20 万随机位置/方向	漏检 0	通过
q±第二测点最坏	扇区采样	807.66 m，统一上界 817.65 m	小于 1000 m，通过
双侧定位单源	3000 随机单源	全部清除	最大 12 次追加测向
双侧定位极端组合	5292 边界组合	全部清除	最大 2 次末端 clear 尝试
122 点光学保底	20 万矩形采样	18.03 m	小于 20 m，通过
直径算法交叉验证	300 随机凸多边形	最大差 0.0 m	通过
最小包围圆	40 m 等边三角形	23.094 m	直径 40 m 不能保证 一次清除
严格外包修复	167358 有效扇形点	0 漏出	通过

示向楔形交会与最小包围圆如图 4 所示。图示中橙色区域为两条方向线约束与目标圆域的交集，绿色圆为覆盖该交集的最小包围圆；只要该圆半径不超过 19.5 米，就可由一次成功的 20 米半径清除覆盖整个可行域。

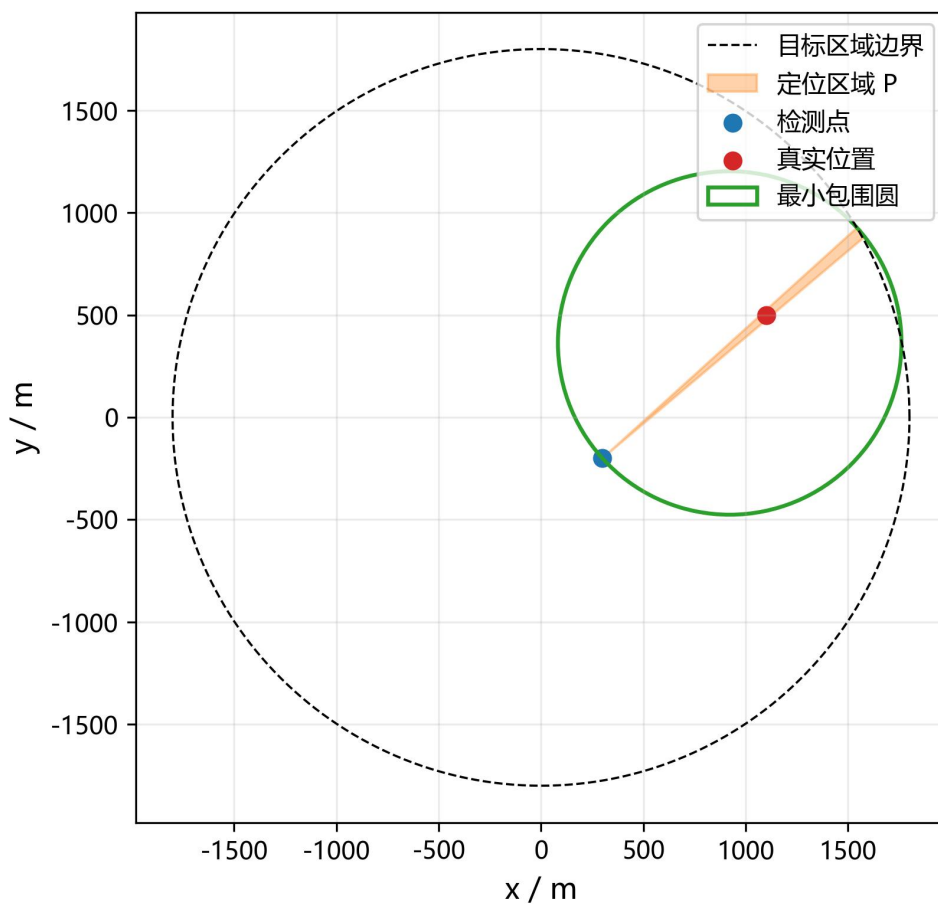


图 4 示向楔形交会与最小包围圆示意图

10.2 本地严格配对实验

在自建规则环境中，对相同种子、相同源位置、频道、半径、类型、方向和固定误差场进行严格配对。四组各 100 对，共 800 局，结果见表 3。

表 3 本地严格配对实验结果

场景	A0 平均(s)	G25O 平均(s)	G25OR 平均(s)	G25OR 相对 A0	G25OR 相对 G25O
Q3 圆内随机	5578.999	4874.028	4219.204	-24.37%	-13.43%
Q4 圆内随机	13303.025	7195.359	7005.831	-47.34%	-2.63%
Q3 边缘压力	4936.612	4763.499	4639.209	-6.02%	-2.61%
Q4 边缘压力	14214.959	9400.996	9399.577	-33.88%	-0.02%

800 局全部清除，G25OR 无光学保底触发，无求解失败。G25O-R 完整版与 G25OR 的四组差异均小于 0.2%，没有实际收益，因此不作为正式方案。

10.3 官方模拟器 G250/G250R/G250-R full 各 100+100 局

三个版本均完成官方问题 3、问题 4 各 100 局自动演练，全部 100%清除，主要指标见表 4。

表 4 官方 100 局演练主要指标

指标	P3 G25O	P3 G25OR	P3 G25O-R full	P4 G25O	P4 G25OR	P4 G25O-R full
全清率	100%	100%	100%	100%	100%	100%
平均虚拟时间/源	319.31 s	328.81 s	318.69 s	575.53 s	556.83 s	558.01 s
中位虚拟时间/源	316.18 s	322.67 s	316.85 s	571.83 s	573.15 s	553.84 s
平均程序运行时间	1.351 s	1.462 s	1.488 s	3.014 s	3.548 s	3.657 s
平均测量次数	108.43	110.39	108.27	282.36	283.33	284.67
平均切频次数	103.50	105.00	103.35	270.91	272.05	273.54
平均失败清除	0.59	0.45	0.44	0.04	0.02	0.03
移动时间占比	82.93%	82.64%	82.83%	75.50%	74.80%	74.82%
反推平均移动距离	17344 m	17206 m	17175 m	26922 m	26021 m	26176 m

两两 Mann-Whitney 检验（平均虚拟时间/源）结果见表 5。

表 5 两两 Mann-Whitney 检验结果

对比	问题 3	问题 4
G25O-R full vs G25O	-0.19%, p=0.931	-3.04%, p=0.225
G25O-R full vs G25OR	-3.08%, p=0.182	+0.21%, p=0.956
G25OR vs G25O	+2.98%, p=0.170	-3.25%, p=0.217

所有差异均不显著。程序运行时间 G25O 最低，但三者都在数秒量级，远低于 20 分钟限制。

三版本平均虚拟时间与平均程序运行时间分别如图 5 和图 6 所示。三个版本的虚拟时间差异远小于程序运行时间差异，说明在该任务中算法计算开销相对于机器狗移动开销可以忽略。

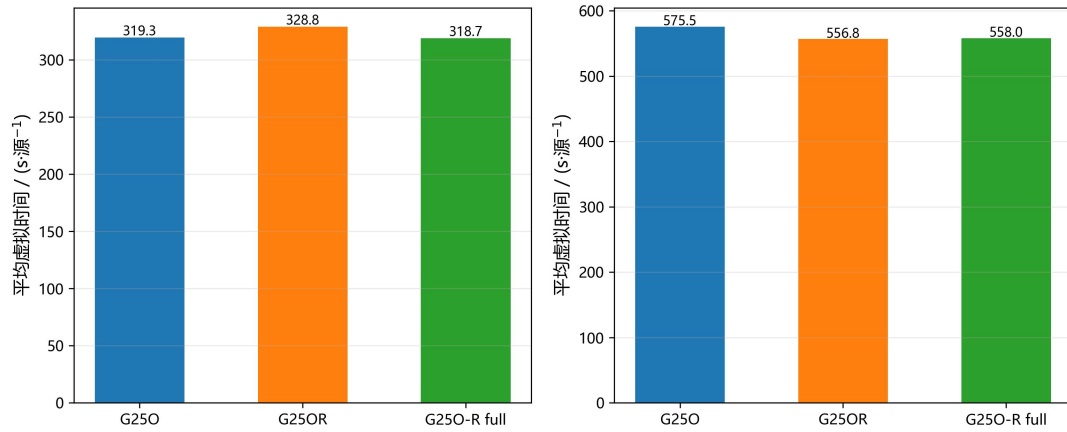


图 5 三个 G25O 变体官方平均虚拟时间/源对比

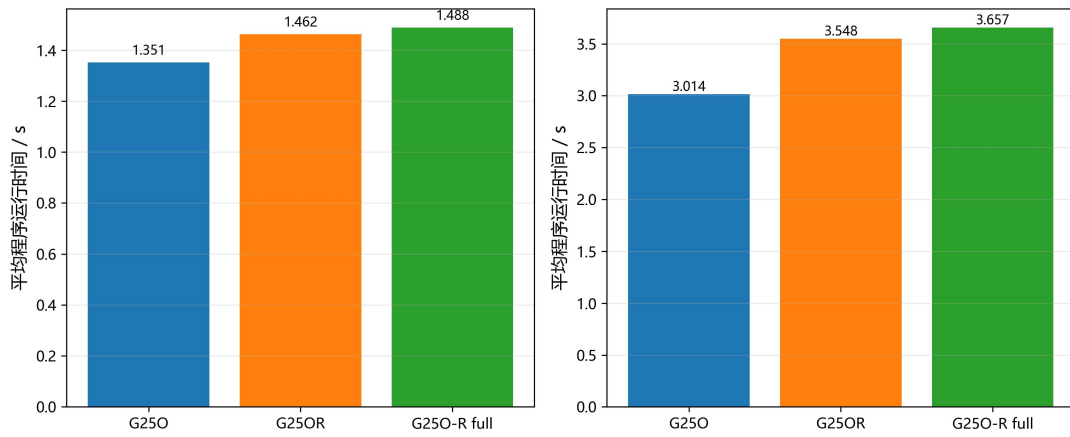


图 6 三个 G25O 变体官方平均程序运行时间对比

10.4 与旧方案 and 传统方法的比较

对比结果见表 6，所有行均给出局数、全清率和时间指标。

表 6 与传统方法和旧方案对比

问题	方案	局数	全清率	平均虚拟时间/源	平均程序运行时间
3	传统方法	14	100%	594.81 s	4.343 s
3	旧安全 PPO	20	100%	428.64 s	2.015 s
3	G25O (100 局)	100	100%	319.31 s	1.351 s
3	G25OR (100 局)	100	100%	328.81 s	1.462 s
4	传统方法 (有效局)	16	100%	1110.58 s	4.710 s
4	旧安全 PPO	20	100%	1122.77 s	5.048 s
4	G25O (100 局)	100	100%	575.53 s	3.014 s

问题	方案	局数	全清率	平均虚拟时间/源	平均程序运行时间
4	G25OR (100局)	100	100%	556.83 s	3.548 s

传统方法问题 4 有一局因 q3-mode-on-q4 模式误跑被剔除，有效传统局全部清除。G25OR 相对旧安全 PPO，问题 3、问题 4 平均虚拟时间/源分别降低约 23.3%和 50.4%；相对有效传统方法分别降低约 44.7%和 49.9%。

与传统方法和旧安全 PPO 的对比见图 7。G25OR 在两个问题上的平均虚拟时间/源均低于传统方法和旧安全 PPO，且程序运行时间保持在数秒量级。

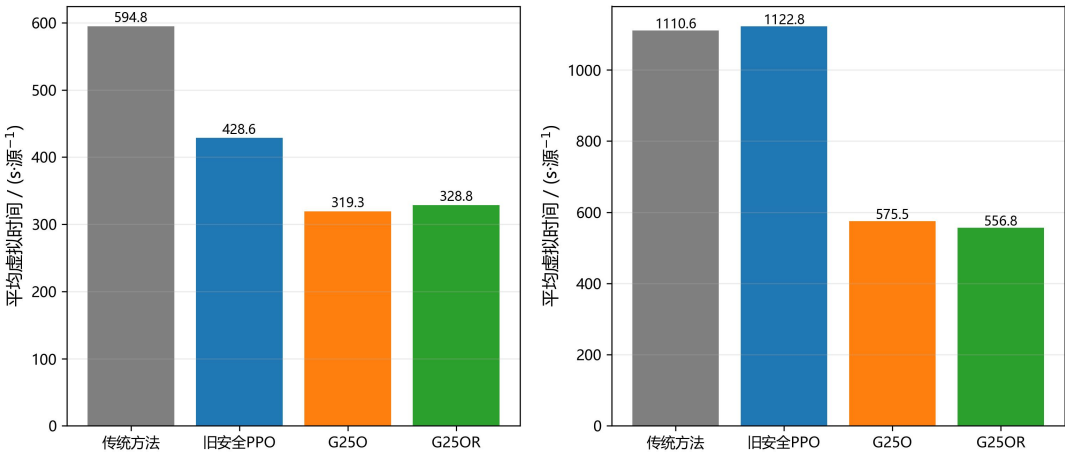


图 7 传统方法与方案 B 平均虚拟时间/源对比

10.5 成本分解与瓶颈

按官方计时公式反推的成本分解见表 7。

表 7 成本分解

项目	问题 3	问题 4
移动时间占比	约 82.9%	约 75.5%
平均移动距离	约 17.2 km	约 26.0-26.9 km
测量、切频、清除等	约 17.1%	约 24.5%

三个版本的移动距离差不超过 1.5%，说明当前瓶颈主要是完成定位和清除所必须的移动路线，而不是测量次数或调度算法。继续优化应研究清除邻域路线、动态覆盖证书和路线级联合调度，而不是继续增加 PPO 训练步数。

10.6 分源数敏感性分析

为检验源数变化对策略的影响，将官方 100 局结果按每局源数分组，三版本的分源数平均虚拟时间/源如图 8 所示。总体趋势是随源数增加，平均单源时间先下降后趋于平稳：源数较少时固定扫描开销占比高；源数增加后，扫描点被多源分摊，单源成本下降；当源数超过约 13 个后，频道账本冲突和重复测量增加，单源成本基本稳定。三个版本在各源数分区间的曲线几乎重合，说明 S25 覆盖和双侧定位已经固定了主要增益，G25OR 的严格外包修复没有以牺牲平均效率为代价。

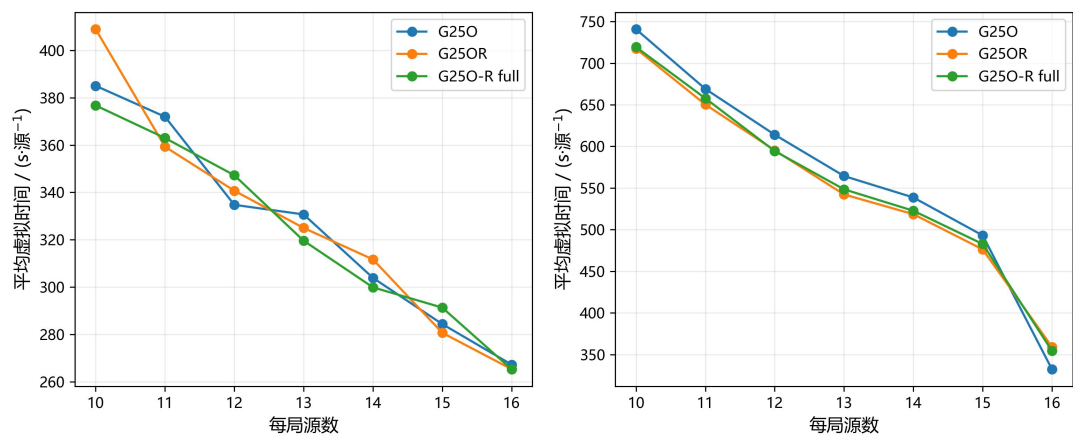


图 8 每局源数敏感性

10.7 正式测试方案与记录表

正式测试尚未执行。建议正式测试采用 G25OR 版本，每题三次机会，测试前冻结参数，不在正式案例上训练或调参。正式测试需填写表 8 格式数据，并从模拟器导出行为日志放入支撑材料，记录表见表 8。

表 8 正式测试记录表

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
问题 3-测试 1	待填	待填	待填
问题 3-测试 2	待填	待填	待填
问题 3-测试 3	待填	待填	待填
问题 4-测试 1	待填	待填	待填
问题 4-测试 2	待填	待填	待填
问题 4-测试 3	待填	待填	待填

十一、模型与算法选择依据

11.1 总体路线选择

本题是部分可观测、硬几何约束与在线决策的混合问题。源位置和数量未知，但接收半径、角度误差、移动速度和清除半径都有明确硬界。候选总体路线有三类，比较见表 9。

表 9 总体路线比较

路线	做法	优点	缺点	是否采用
端到端强化学习	网络直接从观测输出移动和检测动作	通用、可学习	训练慢，完成率无硬保证，容易漏源	否
纯固定规则	固定扫描点+固定清除顺序	简单可解释	路线长，不利用实时信息	否
几何保证+RL 调度	几何层保证覆盖与清除，RL 只决定顺序	正确性可证明，效率可学习	需要两层接口设计	是

选择第三类，因为题目首要目标是确保全部源被清除，不能让完成率依赖奖励函数权重。保证层提供有限步骤完成路径，调度层只优化时间。

11.2 问题 1：为什么选半平面交和最小包围圆

候选定位方法比较见表 10。

表 10 候选定位方法比较

方法	问题	是否采用
两条示向线求交	把带误差的示向线当精确直线，无区域直径概念	否
最小二乘或最大似然	需要统计噪声分布，题目只给硬误差界	否
粒子滤波或贝叶斯	近似后验，无法给出严格清除证书	否
半平面集合估计	计算稍复杂，但精确表示硬误差界	是

选择半平面交，是因为题目给的是 1 度硬误差范围，不是概率分布；多个半平面交集是凸多边形，可直接求直径和最小包围圆。

清除证书方面，候选为直径圆和最小包围圆。选择最小包围圆，因为区域直径为 D 时，以 D 为直径的圆不一定覆盖区域；由 Jung 定理，平面点集最小包围圆半径不超过 $D/\sqrt{3}$ ，等边三角形取等号。只有最小包围圆半径 $r^* \leq 20$ 米才能作为一次清除证书；实现取 $r^* \leq 19.5$ 米，留 0.5 米数值余量。

11.3 问题 2 : 为什么选 Q_{vis} 与 $q_{\pm}$

候选策略包括固定垂直选点、最近可行点、 $q_{\pm}$ 可靠候选和 Q_{vis} 信息评分。固定垂直点可能超出实际接收半径而失联；最近点可能交会角很差。本文先用

$$q_{\pm} = s1 + 750u \pm 300v \quad (25)$$

保证第二点仍能接收：对首次示向整个扇区，统一最坏距离上界 817.65 米，小于最小接收半径 1000 米；密集采样最坏约 807.7 米。再在 $Q_{vis} = \{q: \max_{z \in F} \|q - z\| \leq 1000\}$ 内，用单位时间定位不确定性下降 $G(q)$ 选更优点。这样先保证可靠接收，再追求好的交会几何。

11.4 问题 3、4 : 为什么选 S3 与 S25 解析覆盖

候选覆盖方案包括随机网格、集合覆盖优化和解析构造。随机网格没有严格覆盖证明；集合覆盖优化对未知方向定向源约束复杂，证明和边界处理风险高。本文选择解析构造：

S3 为原点加半径 1200 米正六边形六个顶点，共 7 点；解析最坏发现距离 968.90 米，小于 1000 米。

S25 为原点、内环 12 点、外环 12 点，共 25 点；36 个三角形最大边 975.897 米，小于 1000 米；任意 180 度发射半平面至少包含一个三角形顶点。

S25 完整覆盖路线约 18173.85 米；原 31 点网格任意完整访问下界为 $30 \times 950 = 28500$ 米，因此固定覆盖巡航路线至少缩短 36.2%。优化方法可能找到更少点集，但当前 25 点证明清晰、路线显著短于 31 点基线，风险收益比更好。

11.5 单源清除 : 为什么用双侧定位并保留 122 点保底

候选方法有普通交会、粒子滤波、122 点光学扫描和双侧区间定位。普通交会在定向源失联时几何退化；粒子滤波需要分布假设且无有限步骤保证；122 点光学扫描有有限保证但平均移动较远。双侧区间定位利用两条题设：

定向源有效区域是闭半平面；

最小接收半径有下界 1000 米。

在区间中点两侧成对测量，两次 no_signal 可安全推出真实源位于前半区间；每轮区间至少折半，6 轮后宽度不超过 23.4375 米；末端两点覆盖最坏距离 17.856 米，小于 20 米。因此最多 12 次追加测向加 2 次末端光学尝试即可保证清除。122 点光学扫描不删除，用于接口异常、数值退化或交会失败时的最终保底。

11.6 调度 : 为什么选带动作屏蔽 PPO 并保留 A0

候选调度方法包括 A0 贪心、滚动时域规划、PPO、DQN、SAC 和 MCTS。A0 简单可靠，但短视；滚动规划需要场景采样和较大在线计算；DQN 对持续时间不同

的宏动作折扣处理不自然；SAC 适合连续动作，不适合本题的离散候选点选择；MCTS 需要大量在线模拟。PPO 适合 96 槽位离散宏动作，可通过动作屏蔽禁止明显无效动作，训练相对稳定。

但 PPO 本身不能保证完成，因此本文加入三层保护：
动作屏蔽：禁止已清除频道、无依据局部测量、无义务扫描和无证书退出；
A0 先验与行为克隆：网络早期即保持完成能力；
安全门与接管：PPO 偏离 A0 且优势不足时使用 A0，异常时完全切换 A0。

官方 40 局和本地 100 对实验中 PPO 偏离 A0 次数为 0，因此论文只把 PPO 作为可选调度扩展，不声称它带来速度优势；完成保证由覆盖证书、清除证书和光学保底提供。

11.7 最终版本：为什么选 G25OR

三个候选版本官方各 100+100 局全部清除，对比见表 11。

表 11 G25O 版本对比

版本	主要改动	问题 3 平均时间/ 源	问题 4 平均时间/ 源	结论
G25O	S25+双侧定位	319.31 s	575.53 s	最简单，但示向三角形有 0.233 米外包缺口
G25OR	严格外包 +MEC+滚动插入	328.81 s	556.83 s	严格证明，成绩与最好版本等价
G25O-R full	动态选点+机会测向+滚动清除	318.69 s	558.01 s	最复杂，无显著收益

两两差异均不显著（ $p>0.17$ ）。因此选择 G25OR：它修复了首次示向三角形的严格外包缺口，同时官方成绩与 G25O、G25O-R full 等价；G25O-R full 的额外复杂度和程序运行时间没有换来统计显著的速度提升。

11.8 基线与对比原则

本文设置传统方法、旧安全 PPO、G25O、G25OR 和 G25O-R full 五类对照。本地实验采用相同种子、相同源和相同误差场的严格配对；官方模拟器案例随机生成且不能重放，因此官方数据采用分布比较和 Mann-Whitney 检验，不把非配对差值写成确定的算法增益。评价统一使用总虚拟时间、平均定位清除时间、程序运行时间、清除率和失败场景统计。

十二、结论与模型评价

12.1 主要结论

问题 1 的定位区域可由示向误差半平面交求凸多边形，直径枚举和最小包围圆可分别给出直径和清除证书；直径不超过 40 米不等于一次清除必成功。

问题 2 的 $q_{\pm}$ 候选点具有可证明的接收保证，最坏距离小于 1000 米；结合 Q_{vis} 和信息评分可进一步优选。

问题 3 的 7 点覆盖和问题 4 的 25 点覆盖分别给出全向源和任意 180 度定向源的发现保证；S25 完整覆盖路线比原 31 点布局至少缩短 36.2%。

双侧区间定位把成对 `no_signal` 转化为距离区间收缩，单源最多 12 次追加测向加 2 次末端光学尝试即可保证清除；122 点光学扫描保留为最终保底。

官方 G25O、G25OR、G25O-R full 各 100+100 局全部清除，三版本平均时间无显著差异；G25OR 在保持数学严格外包修复的同时成绩等价，因此作为最终方案。

PPO 在当前实现中没有偏离 A0 先验，未独立产生速度收益；强化学习适合作为上层调度框架和消融对象，完成保证必须由确定性几何层提供。

12.2 模型优点

完成条件由覆盖证书和有限保底给出，不依赖奖励函数偏好。

几何层与决策层分离，便于验证和公平对比。

25 点覆盖和双侧定位均有解析证明，辅以大规模数值核验。

官方演练样本达到每版本 100+100 局，结论具有分布统计支撑。

明确区分本地仿真数据、官方演练数据和尚未执行的正式测试，不伪造结果。

12.3 不足与改进

正式测试尚未执行，表 8 需要有官方正式数据后填写。

当前移动占比仍高达 75%-83%，路线级优化空间最大。

RL 网络在安全先验下偏离率低，未能发挥学习调度优势；可研究更弱的先验和更好的候选动作结构，但必须保证完成证书。

Q4 动态无源凸包证书、清除邻域 TSPN 和机会式试清仍处于设计或初步实验阶段。官方案例不能重放，因此官方对比是分布比较而非同案例配对。

参考文献

- [1] 2026 年高教社杯全国大学生数学建模竞赛 B 题：无线电干扰源的快速自动定位与清除。
- [2] 全国大学生数学建模竞赛 B 题附件 1：模拟器使用说明。
- [3] 全国大学生数学建模竞赛 B 题附件 2：模拟器通信接口说明及编程指南。
- [4] Schulman J, Wolski F, Dhariwal P, et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
- [5] Sutton R S, Precup D, Singh S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence, 1999, 112(1-2):181-211.
- [6] Huang S, Ontanon S. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. arXiv:2006.14171, 2020.
- [7] Welzl E. Smallest enclosing disks (balls and ellipsoids). New Results and New Trends in Computer Science, 1991:359-370.

附录 A 代码与复现

项目代码主要模块见表 12，完整可运行源码见支撑材料代码目录。

表 12 代码模块清单

模块	作用
geometry.py	半平面、凸多边形裁剪、直径、最小包围圆
coverage.py	S3、S4、S25 覆盖点构造与核验
bilateral.py	双侧区间定位与末端光学清除
g25o.py	G25O、G25OR、G25O-R 完整版策略
local_env.py	本地规则环境与宏动作
policy.py、ppo.py	带动作屏蔽 PPO、A0 先验与训练
remote.py	官方 HTTP+JSON 适配器
scripts/run_official_g25o.py	官方演练入口
scripts/auto_official_g25o_batch.py	自动批量演练入口
scripts/report_three_g25_variants.py	三方结果汇总

复现顺序：

```
python scripts/verify_math.py
python scripts/run_g25o_compare.py --n 100 --start 92000 --variant G25OR
python scripts/run_official_g25o.py --mode 3 --robot-id <队号> --variant G25OR
python scripts/report_three_g25_variants.py
```

A.2 核心源代码

以下列出与保证性、定位和清除证书直接相关的核心模块代码；完整工程见支撑材料中的代码目录。

A.2.1 coverage.py : S3 与 S25 覆盖构造

```
"""保证发现覆盖的测点集：问题 3 的 S3 与问题 4 的 S4."""
from __future__ import annotations

from math import cos, pi, sin, sqrt
from typing import List

import numpy as np

DOMAIN_RADIUS = 1800.0
MIN_RECEIVE_RADIUS = 1000.0

def s3_points(hex_radius: float = 1200.0) -> np.ndarray:
    """原点 + 半径 1200 的正六边形六个顶点，共 7 点。"""
    pts = [np.array([0.0, 0.0])]
    for k in range(6):
        a = k * pi / 3.0
        pts.append(np.array([hex_radius * cos(a), hex_radius * sin(a)]))
    return np.asarray(pts, dtype=float)
```

```

def triangular_lattice(h: float = 950.0, limit: float = DOMAIN_RADIUS + 950.0) -> np.ndarray:
    """三角网格点 s_ij=(h(i+j/2), sqrt(3)/2*h*j), 保留 ||s|| <= limit."""
    # 先估计整数范围
    kmax = int(limit / max(h, 1e-9)) + 4
    pts = []
    for j in range(-kmax, kmax + 1):
        for i in range(-kmax, kmax + 1):
            x = h * (i + 0.5 * j)
            y = (sqrt(3.0) / 2.0) * h * j
            if x * x + y * y <= limit * limit + 1e-9:
                pts.append((x, y))
    # 去重并排序 (原点优先, 便于固定槽位)
    uniq = []
    for p in pts:
        if not any((p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2 < 1e-12 for q in uniq):
            uniq.append(p)
    return np.asarray(uniq, dtype=float)

def s4_points(h: float = 950.0) -> np.ndarray:
    return triangular_lattice(h=h, limit=DOMAIN_RADIUS + h)

def coverage_verify_s3(samples: int = 200000, seed: int = 1) -> dict:
    """数值核验 S3 对全向源的发现覆盖。"""
    rng = np.random.default_rng(seed)
    pts = s3_points()
    # 圆内均匀采样 + 边界采样
    n1 = samples * 3 // 4
    r = DOMAIN_RADIUS * np.sqrt(rng.random(n1))
    th = rng.random(n1) * 2 * pi
    q = np.column_stack([r * np.cos(th), r * np.sin(th)])
    n2 = samples - n1
    th2 = rng.random(n2) * 2 * pi
    q2 = np.column_stack([DOMAIN_RADIUS * np.cos(th2), DOMAIN_RADIUS * np.sin(th2)])
    q = np.vstack([q, q2])
    worst = 0.0
    miss = 0
    for p in q:
        d = np.linalg.norm(pts - p, axis=1).min()
        worst = max(worst, float(d))
        if d > MIN_RECEIVE_RADIUS:
            miss += 1
    return {"points": len(pts), "worst_distance": worst, "misses": miss, "samples": len(q)}

def coverage_verify_s4(samples: int = 200000, seed: int = 2) -> dict:
    """数值核验 S4 对任意 180 度定向源的发现覆盖 (每点随机一个方向)。"""
    rng = np.random.default_rng(seed)
    pts = s4_points()
    n1 = samples * 3 // 4
    r = DOMAIN_RADIUS * np.sqrt(rng.random(n1))
    th = rng.random(n1) * 2 * pi
    q = np.column_stack([r * np.cos(th), r * np.sin(th)])
    n2 = samples - n1
    th2 = rng.random(n2) * 2 * pi
    q2 = np.column_stack([DOMAIN_RADIUS * np.cos(th2), DOMAIN_RADIUS * np.sin(th2)])
    q = np.vstack([q, q2])
    dirs = rng.random((len(q), 2)) * 2 - 1
    dirs /= np.linalg.norm(dirs, axis=1, keepdims=True) + 1e-15
    worst_visible = 0.0
    miss = 0
    for p, n in zip(q, dirs):
        rel = pts - p
        dist = np.linalg.norm(rel, axis=1)
        visible = (rel @ n) >= -1e-9
        if not np.any(visible & (dist <= MIN_RECEIVE_RADIUS)):

```

```

        miss += 1
    else:
        # 记录在可见点中的最小距离最大值
        dv = dist[visible]
        if len(dv):
            worst_visible = max(worst_visible, float(dv.min()))
    return {"points": len(pts), "misses": miss, "samples": len(q), "worst_visible_min":
worst_visible}

if __name__ == "__main__":
    p3 = s3_points()
    p4 = s4_points()
    print("S3", len(p3), p3.tolist())
    print("S4", len(p4))
    print(coverage_verify_s3(samples=50000))
    print(coverage_verify_s4(samples=50000))

def s25_points() -> np.ndarray:
    """问题 4 新覆盖构造：原点 + 内环 12 点 + 外环 12 点，共 25 点。

    内环半径 970m，角度 15+30k 度；外环半径 1880m，角度 30k 度。
    外正十二边形内切圆半径  $1880 \cdot \cos 15^\circ = 1815.94\text{m} > 1800\text{m}$ ，覆盖目标圆。
    36 个三角形最大边长约  $975.90\text{m} < 1000\text{m}$ ，保证任意 180 度定向源至少一点可见。
    """
    k = np.arange(12, dtype=float)
    origin = np.zeros((1, 2), dtype=float)
    a_in = np.deg2rad(15.0 + 30.0 * k)
    inner = 970.0 * np.column_stack([np.cos(a_in), np.sin(a_in)])
    a_out = np.deg2rad(30.0 * k)
    outer = 1880.0 * np.column_stack([np.cos(a_out), np.sin(a_out)])
    return np.vstack([origin, inner, outer])

def s25_max_triangle_edge() -> float:
    """返回 36 个覆盖三角形的最大边长。"""
    pts = s25_points()
    0 = pts[0]
    inner = pts[1:13]
    outer = pts[13:25]
    edges = []
    for k in range(12):
        k2 = (k + 1) % 12
        tri1 = [0, inner[k], inner[k2]]
        tri2 = [outer[k], outer[k2], inner[k]]
        tri3 = [inner[k], inner[k2], outer[k2]]
        for tri in (tri1, tri2, tri3):
            for i in range(3):
                edges.append(float(np.linalg.norm(tri[i] - tri[(i + 1) % 3])))
    return max(edges)

```

A.2.2 bilateral.py：双侧区间定位

"""双侧区间定位：把定向源的成对 no_signal 转化为距离区间收缩。

该模块只使用 measure/clear 回调，不访问真实源。证明与上界来自新方案论证：
首次有效定向后最多 12 次追加测向，末端最多 2 次光学 clear。

```

"""
from __future__ import annotations

import math
from dataclasses import dataclass
from typing import Callable, Optional

import numpy as np

from .geometry import minimum_enclosing_circle

```

```

EPS_DEG = 1.01 # 1度传感误差 + 0.005度两位小数舍入余量
EPS = math.radians(EPS_DEG)
K = math.tan(EPS)
R_MAX = 1500.0

def clip(poly: np.ndarray, normal: np.ndarray, offset: float) -> np.ndarray:
    """半平面 normal @ z <= offset 的凸多边形裁剪。"""
    if poly is None or len(poly) == 0:
        return np.empty((0, 2), dtype=float)
    poly = np.asarray(poly, dtype=float)
    vals = poly @ normal - offset
    out = []
    n = len(poly)
    for i, a in enumerate(poly):
        b = poly[(i + 1) % n]
        da = float(vals[i]); db = float(vals[(i + 1) % n])
        if da <= 1e-9:
            out.append(a)
        if (da < -1e-9 and db > 1e-9) or (da > 1e-9 and db < -1e-9):
            out.append(a + (b - a) * (da / (da - db)))
    if not out:
        return np.empty((0, 2), dtype=float)
    return np.asarray(out, dtype=float).reshape(-1, 2)

def bearing_clip(poly: np.ndarray, p: np.ndarray, theta: float) -> np.ndarray:
    u = np.array([math.cos(theta), math.sin(theta)], dtype=float)
    v = np.array([-u[1], u[0]], dtype=float)
    for normal in (-u, v - K * u, -v - K * u):
        poly = clip(poly, normal, float(normal @ p))
    return poly

@dataclass
class SolveResult:
    success: bool
    extra_measures: int
    clear_attempts: int
    rounds: int
    position: np.ndarray
    region: np.ndarray

def solve_bilateral(first_position, first_bearing_deg: float, current_position,
                    measure: Callable, clear: Callable, *,
                    geometry: bool = True,
                    initial_region: Optional[np.ndarray] = None) -> SolveResult:
    """用双侧检测解决一个已确认源。

    measure(q)->{'measure_result':..., 'svd_deg':...}
    clear(q)->{'clear_result':...}
    回调必须已绑定频道, 并校验 accepted 与结果字段。
    """
    origin = np.asarray(first_position, dtype=float)
    a0 = math.radians(float(first_bearing_deg))
    u = np.array([math.cos(a0), math.sin(a0)], dtype=float)
    v = np.array([-u[1], u[0]], dtype=float)
    B = np.stack([u, v], axis=1)
    pos = np.asarray(current_position, dtype=float).copy()
    lo, hi = 0.0, R_MAX
    poly = np.array([[0.0, 0.0], [R_MAX, -R_MAX * K], [R_MAX, R_MAX * K]], dtype=float)
    if initial_region is not None:
        supplied = (np.asarray(initial_region, dtype=float) - origin) @ B
        for normal, c in ((np.array([-1.0, 0.0]), 0.0), (np.array([1.0, 0.0]), R_MAX),
                        (np.array([-K, 1.0]), 0.0), (np.array([-K, -1.0]), 0.0)):
            supplied = clip(supplied, normal, c)

```

```

    if len(supplied) == 0:
        raise RuntimeError("Invalid initial conservative region")
    poly = supplied
    lo = max(lo, float(poly[:, 0].min()))
    hi = min(hi, float(poly[:, 0].max()))
nm = nc = rounds = 0

def glob(z):
    return origin + B @ np.asarray(z, dtype=float)

def ret(ok):
    return SolveResult(ok, nm, nc, rounds, pos.copy(), poly @ B.T + origin)

for _ in range(7):
    if geometry:
        lo = max(lo, float(poly[:, 0].min()))
        hi = min(hi, float(poly[:, 0].max()))
        mec = minimum_enclosing_circle(poly)
        if float(mec.radius) <= 19.5:
            pos = glob(mec.center)
            nc += 1
            ans = clear(pos)
            if ans.get("clear_result") == "success":
                return ret(True)
            raise RuntimeError("Certified optical clear failed")
    if hi - lo <= 24.0 + 1e-7:
        break
    rounds += 1
    mid = (lo + hi) / 2.0
    side = mid * K + 5.0
    probes = [np.array([mid, side]), np.array([mid, -side])]
    probes.sort(key=lambda q: float(np.linalg.norm(glob(q) - pos))
    observed = False
    for q in probes:
        pos = glob(q)
        nm += 1
        ans = measure(pos)
        status = ans.get("measure_result")
        if status == "near":
            nc += 1
            if clear(pos).get("clear_result") == "success":
                return ret(True)
            raise RuntimeError("near clear failed")
        if status == "no_signal":
            continue
        if status != "direction" or "svd_deg" not in ans:
            raise RuntimeError(f"Invalid measure response: {ans}")
        observed = True
        theta = math.radians(float(ans["svd_deg"]) - first_bearing_deg)
        cx = math.cos(theta)
        if cx > math.sin(EPS) + 1e-12:
            lo = max(lo, mid)
        elif cx < -math.sin(EPS) - 1e-12:
            hi = min(hi, mid)
        else:
            w = (R_MAX * K + side) * math.tan(2 * EPS)
            lo = max(lo, mid - w)
            hi = min(hi, mid + w)
        if geometry:
            poly = bearing_clip(poly, q, theta)
        break
    if not observed:
        # 双侧 no_signal: 若 x>=mid, 则两探测点连线上存在首次可见点与源之间的点,
        # 且两探测点都比首次测点更近, 不可能同时不可见, 故 x<mid.
        hi = min(hi, mid)
    poly = clip(poly, np.array([1.0, 0.0]), hi)
    poly = clip(poly, np.array([-1.0, 0.0]), -lo)
    if len(poly) == 0 or lo > hi + 1e-7:

```

```

        raise RuntimeError("Empty certified region")
    if hi - lo > 24.0 + 1e-6:
        raise RuntimeError("Interval did not contract within six rounds")
    targets = [np.array([(lo + hi) / 2.0, hi * K / 2.0]),
               np.array([(lo + hi) / 2.0, -hi * K / 2.0])]
    targets.sort(key=lambda q: float(np.linalg.norm(glob(q) - pos)))
    for q in targets:
        pos = glob(q)
        nc += 1
        if clear(pos).get("clear_result") == "success":
            return ret(True)
    raise RuntimeError("Two-disk terminal cover failed")

```

A.2.3 geometry.py : 半平面交、直径与最小包围圆核心函数

```

class HalfPlane:
    """a*x + b*y <= c."""
    a: float
    b: float
    c: float

    def normalized(self) -> "HalfPlane":
        n = (self.a * self.a + self.b * self.b) ** 0.5
        if n < EPS:
            raise ValueError("退化半平面")
        return HalfPlane(self.a / n, self.b / n, self.c / n)

    def signed_distance(self, p: np.ndarray) -> float:
        return float(self.a * p[0] + self.b * p[1] - self.c)

def unit_from_deg(theta_deg: float) -> np.ndarray:
    t = np.deg2rad(theta_deg)
    return np.array([cos(t), sin(t)], dtype=float)

def perpendicular(v: np.ndarray) -> np.ndarray:
    return np.array([-v[1], v[0]], dtype=float)

def wedge_halfplanes(s: Sequence[float], theta_deg: float, eps_deg: float = SVD_ERROR_DEG) ->
List[HalfPlane]:
    """由检测点 s 与示向度 theta 构造误差楔形的半平面表示。

     $u^T(z-s) \geq 0$ 
     $|v^T(z-s)| \leq \tan(\epsilon) u^T(z-s)$ 
    """
    s = np.asarray(s, dtype=float)
    u = unit_from_deg(theta_deg)
    v = perpendicular(u)
    t = tan(np.deg2rad(eps_deg))
    # forward:  $u^T z \geq u^T s \Rightarrow (-u)^T z \leq -u^T s$ 
    hps = [HalfPlane(-u[0], -u[1], -float(np.dot(u, s)))]
    #  $v^T(z-s) \leq t u^T(z-s)$ 
    n1 = v - t * u
    hps.append(HalfPlane(n1[0], n1[1], float(np.dot(n1, s))))
    #  $-v^T(z-s) \leq t u^T(z-s)$ 
    n2 = -v - t * u
    hps.append(HalfPlane(n2[0], n2[1], float(np.dot(n2, s))))
    return [hp.normalized() for hp in hps]

def circle_outer_polygon(center: Sequence[float], radius: float, n: int = 96) -> np.ndarray:
    """圆的外切正 n 边形 (包含圆), 用于保守近似圆形约束。"""
    c = np.asarray(center, dtype=float)
    k = np.arange(n, dtype=float)
    # 外切正 n 边形顶点半径 r / cos(pi/n)
    R = radius / cos(pi / n)

```

```

    ang = 2.0 * pi * k / n
    return c[None, :] + R * np.column_stack([np.cos(ang), np.sin(ang)])

def circle_outer_halfplanes(center: Sequence[float], radius: float, n: int = 96) ->
List[HalfPlane]:
    """圆的外切正 n 边形约束，作为半平面集合。"""
    c = np.asarray(center, dtype=float)
    hps: List[HalfPlane] = []
    for k in range(n):
        ang = 2.0 * pi * k / n
        normal = np.array([cos(ang), sin(ang)])
        hps.append(HalfPlane(normal[0], normal[1], float(np.dot(normal, c) +
radius)).normalized())
    return hps

def domain_polygon(radius: float = DOMAIN_RADIUS, n: int = 128) -> np.ndarray:
    return circle_outer_polygon((0.0, 0.0), radius, n=n)

def poly_signed_area(poly: np.ndarray) -> float:
    if poly is None or len(poly) < 3:
        return 0.0
    x = poly[:, 0]
    y = poly[:, 1]
    return 0.5 * float(np.sum(x * np.roll(y, -1) - y * np.roll(x, -1)))

def clip_convex_polygon(poly: np.ndarray, hp: HalfPlane) -> np.ndarray:
    """Sutherland-Hodgman 单半平面裁剪。"""
    if poly is None or len(poly) == 0:
        return np.empty((0, 2), dtype=float)
    hp = hp.normalized()
    n = len(poly)
    if n == 1:
        return poly if hp.signed_distance(poly[0]) <= EPS else np.empty((0, 2), dtype=float)
    out: List[np.ndarray] = []
    for i in range(n):
        p = poly[i]
        q = poly[(i + 1) % n]
        fp = hp.signed_distance(p)
        fq = hp.signed_distance(q)
        pin = fp <= EPS
        qin = fq <= EPS
        if pin:
            out.append(p)
        if pin != qin:
            denom = fp - fq
            if abs(denom) > 1e-15:
                t = fp / denom
                out.append(p + t * (q - p))
    if not out:
        return np.empty((0, 2), dtype=float)
    # 去重
    res = []
    for p in out:
        if not res or np.linalg.norm(p - res[-1]) > 1e-9:
            res.append(p)
    if len(res) > 1 and np.linalg.norm(res[0] - res[-1]) <= 1e-9:
        res.pop()
    return np.asarray(res, dtype=float)

def intersect_halfplanes(halfplanes: Sequence[HalfPlane], initial: Optional[np.ndarray] =
None,
                        add_domain: bool = True, domain_radius: float = DOMAIN_RADIUS) ->
np.ndarray:

```

```

"""求半平面交。默认先与目标圆域的外切多边形求交，保证有界。

返回凸多边形顶点；无交集时返回空数组。
"""
if initial is None:
    if add_domain:
        poly = domain_polygon(domain_radius)
    else:
        # 大框仅用于最后防数值溢出；调用方不应把它误当题目约束。
        B = 1_000_000.0
        poly = np.array([[-B, -B], [B, -B], [B, B], [-B, B]], dtype=float)
else:
    poly = np.asarray(initial, dtype=float)
for hp in halfplanes:
    poly = clip_convex_polygon(poly, hp)
    if len(poly) == 0:
        break
return poly

def polygon_diameter(poly: np.ndarray) -> float:
    """区域直径，第一版枚举顶点对。"""
    if poly is None or len(poly) < 2:
        return 0.0
    pts = np.asarray(poly)
    dmax = 0.0
    n = len(pts)
    for i in range(n):
        d = np.linalg.norm(pts[i + 1:] - pts[i], axis=1)
        if len(d):
            dmax = max(dmax, float(np.max(d)))
    return dmax

def polygon_diameter_rotating_calipers(poly: np.ndarray) -> float:
    """旋转卡壳求凸多边形直径，用于与暴力顶点对结果交叉验证。

    输入多边形应为逆时针凸多边形。算法在凸多边形上维护对踵点，
    时间复杂度  $O(n)$ ， $n$  为顶点数。
    """
    if poly is None or len(poly) < 2:
        return 0.0
    pts = np.asarray(poly, dtype=float)
    n = len(pts)
    if n == 2:
        return float(np.linalg.norm(pts[0] - pts[1]))
    k = 1
    best = 0.0
    for i in range(n):
        if k == i:
            k = (k + 1) % n
        while True:
            nk = (k + 1) % n
            if nk == i:
                break
            cur = float(np.linalg.norm(pts[i] - pts[k]))
            nxt = float(np.linalg.norm(pts[i] - pts[nk]))
            if nxt > cur:
                k = nk
            else:
                break
        best = max(best, float(np.linalg.norm(pts[i] - pts[k])))
    return best

def verify_diameter_crosscheck(poly: np.ndarray, tol: float = 1e-6) -> Tuple[float, float, float]:

```

```

"""返回 (暴力直径, 旋转卡壳直径, 绝对差)。用于质量 Gate。"""
d1 = polygon_diameter(poly)
d2 = polygon_diameter_rotating_calipers(poly)
return d1, d2, abs(d1 - d2)

def polygon_centroid(poly: np.ndarray) -> np.ndarray:
    if poly is None or len(poly) == 0:
        return np.zeros(2, dtype=float)
    if len(poly) == 1:
        return poly[0].copy()
    if len(poly) == 2:
        return 0.5 * (poly[0] + poly[1])
    area = poly_signed_area(poly)
    if abs(area) < 1e-12:
        return poly.mean(axis=0)
    cx = cy = 0.0
    n = len(poly)
    for i in range(n):
        x0, y0 = poly[i]
        x1, y1 = poly[(i + 1) % n]
        cross = x0 * y1 - x1 * y0
        cx += (x0 + x1) * cross
        cy += (y0 + y1) * cross
    return np.array([cx, cy], dtype=float) / (6.0 * area)

def point_in_convex_polygon(p: Sequence[float], poly: np.ndarray, tol: float = 1e-7) -> bool:
    if poly is None or len(poly) == 0:
        return False
    p = np.asarray(p, dtype=float)
    n = len(poly)
    if n == 1:
        return np.linalg.norm(p - poly[0]) <= tol
    if n == 2:
        a, b = poly
        ab = b - a
        t = np.dot(p - a, ab) / max(np.dot(ab, ab), 1e-15)
        if t < -tol or t > 1 + tol:
            return False
        return np.linalg.norm(a + np.clip(t, 0, 1) * ab - p) <= tol
    sign = 0.0
    for i in range(n):
        a = poly[i]
        b = poly[(i + 1) % n]
        cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
        if abs(cross) <= tol:
            continue
        s = 1.0 if cross > 0 else -1.0
        if sign == 0:
            sign = s
        elif sign != s:
            return False
    return True

# ----- 最小包围圆 -----
@dataclass
class Circle:
    center: np.ndarray
    radius: float

def circle_from_two(a: np.ndarray, b: np.ndarray) -> Circle:
    c = 0.5 * (a + b)
    return Circle(c, float(np.linalg.norm(a - b) * 0.5))

```

```

def circle_from_three(a: np.ndarray, b: np.ndarray, c: np.ndarray) -> Circle:
    # 外接圆
    ax, ay = a
    bx, by = b
    cx, cy = c
    d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
    if abs(d) < 1e-15:
        # 共线: 取最远两点
        pts = [a, b, c]
        best = Circle(a.copy(), 0.0)
        for i in range(3):
            for j in range(i + 1, 3):
                cc = circle_from_two(pts[i], pts[j])
                if cc.radius > best.radius:
                    best = cc
        return best
    ux = ((ax * ax + ay * ay) * (by - cy) + (bx * bx + by * by) * (cy - ay) + (cx * cx + cy *
cy) * (ay - by)) / d
    uy = ((ax * ax + ay * ay) * (cx - bx) + (bx * bx + by * by) * (ax - cx) + (cx * cx + cy *
cy) * (bx - ax)) / d
    center = np.array([ux, uy], dtype=float)
    return Circle(center, float(np.linalg.norm(center - a)))

def _welzl(points: np.ndarray, boundary: List[np.ndarray], n: int) -> Circle:
    if n == 0 or len(boundary) == 3:
        if len(boundary) == 0:
            return Circle(np.zeros(2), 0.0)
        if len(boundary) == 1:
            return Circle(boundary[0].copy(), 0.0)
        if len(boundary) == 2:
            return circle_from_two(boundary[0], boundary[1])
        return circle_from_three(boundary[0], boundary[1], boundary[2])
    p = points[n - 1]
    d = _welzl(points, boundary, n - 1)
    if np.linalg.norm(p - d.center) <= d.radius + 1e-8:
        return d
    return _welzl(points, boundary + [p.copy()], n - 1)

def minimum_enclosing_circle(points: Sequence[Sequence[float]]) -> Circle:
    """Welzl 随机增量算法, 返回覆盖点集的最小圆近似。"""
    pts = np.asarray(points, dtype=float)
    if pts.size == 0:
        return Circle(np.zeros(2), 0.0)
    if pts.shape[0] == 1:
        return Circle(pts[0].copy(), 0.0)
    # 去重
    uniq = []
    for p in pts:
        if not uniq or np.linalg.norm(p - uniq[-1]) > 1e-9:
            uniq.append(p)
    if len(uniq) < len(pts):
        # 不只比较相邻, 简单做一轮去重
        seen = []
        for p in pts:
            if all(np.linalg.norm(p - q) > 1e-9 for q in seen):
                seen.append(p)
        pts = np.asarray(seen, dtype=float)
    rng = np.random.default_rng(20260911)
    pts = pts[rng.permutation(len(pts))]
    c = _welzl(pts, [], len(pts))
    # 数值微调半径
    if len(pts):
        c.radius = max(c.radius, float(np.max(np.linalg.norm(pts - c.center, axis=1))))
    return c

```

```
def conservative_clear_certificate(poly: np.ndarray, margin: float = 19.5) -> Tuple[bool,
Optional[np.ndarray], float]:
    """检查是否可一次 clear 成功; 返回 (是否, 清除点, 最小包围圆半径)。"""
    if poly is None or len(poly) == 0:
        return False, None, float("inf")
    c = minimum_enclosing_circle(poly)
    return c.radius <= margin, c.center, float(c.radius)
```

A.2.4 g25o.py : 开路路线、严格外包与清除点选择核心函数

```
def route_open(points: np.ndarray, start: Sequence[float]) -> List[int]:
    pts = np.asarray(points, dtype=float)
    ps = np.vstack([np.asarray(start, dtype=float).reshape(1, 2), pts])
    D = np.linalg.norm(ps[:, None, :] - ps[None, :, :], axis=-1)
    rem = set(range(1, len(ps)))
    seq = [0]
    while rem:
        i = min(rem, key=lambda k: (D[seq[-1], k], k))
        seq.append(i)
        rem.remove(i)
    for _ in range(30):
        improved = False
        for i in range(1, len(seq) - 1):
            for j in range(i + 1, len(seq)):
                old = D[seq[i - 1], seq[i]]
                new = D[seq[i - 1], seq[j]]
                if j + 1 < len(seq):
                    old += D[seq[j], seq[j + 1]]
                    new += D[seq[i], seq[j + 1]]
                if new < old - 1e-8:
                    seq[i:j + 1] = seq[i:j + 1][::-1]
                    improved = True
        if not improved:
            break
    return [i - 1 for i in seq[1:]]

def initial_track_polygon(p: np.ndarray, theta_deg: float) -> np.ndarray:
    # 严格外包半径 1500m 的扇形: 用远边弦在中央方向投影为 1500m。
    # 两个边界点取  $p + (1500/\cos \epsilon) * u(\theta \pm \epsilon)$ , 三角形包含整个扇形。
    eps = math.radians(1.01)
    th = math.radians(theta_deg)
    L = 1500.0 / math.cos(eps)
    u1 = np.array([math.cos(th - eps), math.sin(th - eps)])
    u2 = np.array([math.cos(th + eps), math.sin(th + eps)])
    poly = np.asarray([p, p + L * u1, p + L * u2], dtype=float)
    return _clip_target_square(poly)

def _best_clear_point(P: np.ndarray, A: Sequence[float], B: Sequence[float],
                    margin: float = 0.35) -> Optional[np.ndarray]:
    """在保证一次 clear 成功的区域内, 找使 A->q->B 绕行最小的清除点。

    若可行域最小包围圆半径  $r \leq 19.5$ , 则圆  $B(c, 20 - r - \text{margin})$  内任意 q
    都保证与 P 中任意点距离  $\leq 20$ 。返回 None 表示当前没有清除证书。
    """
    c, r = _poly_center_radius(P)
    if r > 19.5:
        return None
    slack = 20.0 - r - margin
    if slack <= 1e-9:
        return None
    A = np.asarray(A, dtype=float)
    B = np.asarray(B, dtype=float)
    AB = B - A
    L = float(np.linalg.norm(AB))
    cand = [c]
    if L > 1e-9:
        t = float(np.dot(c - A, AB) / (L * L))
```

```
t = min(1.0, max(0.0, t))
proj = A + t * AB
v = proj - c
nv = float(np.linalg.norm(v))
if nv <= slack:
    cands.append(proj)
else:
    cands.append(c + v / nv * slack)
valid = [q for q in cands if float(np.linalg.norm(q - c)) <= slack + 1e-9]
if not valid:
    return None
return min(valid, key=lambda q: float(np.linalg.norm(A - q) + np.linalg.norm(q - B)))
```

A.3 支撑材料清单

支撑材料清单见表 13。

表 13 支撑材料清单

文件或目录	内容
代码/brl	几何、覆盖、双侧定位、G25O、PPO、官方接口等完整可运行源码
代码/scripts	数学校验、批量演练、官方 100 局自动运行和结果汇总脚本
数据记录	官方 G25O/G25OR/G25O-R full 各 100 局汇总、传统对比、数学校验和显著性与敏感性数据
AI 工具使用详情.md	AI 工具使用范围、人工核验方式和责任声明（独立支撑材料）
README.md、SKILL_应用说明.md	项目复现说明与技能使用说明
B 题_方案 B_提交包.zip	上述材料打包文件